AgentGuard Project Guide

A plain-language guide to running, deploying, and explaining the project

GOVERNANCE / MCP TOOLS / ANOMALY DETECTION / HUMAN REVIEW

This guide explains AgentGuard in plain language, how to run it, and how to prepare it for GitHub and Vercel. The local Docker setup is the reliable first run. Vercel hosts the Next.js dashboard; the Spring backend, ML scorer, MCP tool server, and PostgreSQL database need separate hosting.

What AgentGuard does

AgentGuard puts a security checkpoint in front of tools used by AI agents. An agent identifies itself with an API key and asks the Spring Boot service to call a tool. The service checks policy, records the event, scores behavior, and forwards permitted calls. Suspicious activity creates a case. A human reviews cases and decisions. The language model can explain evidence, but it cannot approve or run containment actions.

Architecture in five parts

1. **Dashboard:** Next.js pages for login, overview, agents, cases, policies, and audit history.
2. **Governance API:** Spring Boot and Java 21 handle accounts, roles, agent keys, policies, approvals, audit records, and the tool proxy.
3. **Tool service:** FastAPI exposes `web_search`, `db_query`, `file_read`, `file_write`, and `code_exec` behind the proxy. The backend and tool service share an internal token; code execution has process resource limits, but network isolation still needs to be added before running untrusted code in production.
4. **Anomaly scorer:** Python, FastAPI, and scikit-learn run a per-agent Isolation Forest. The demo trains from synthetic data; real deployments need representative data.
5. **Database and investigation:** PostgreSQL stores platform data; pgvector stores embeddings. Ollama or a free-tier Groq/Gemini key can draft investigation text. Code checks event citations, then a human decides.

Tool call flow: `agent + API key -> Spring policy check -> anomaly score -> MCP tool -> event and audit record`.

DENY blocks the tool and names the policy rule. APPROVE waits for a human. An anomaly over the agent's threshold opens a case. If ML is unavailable, high-sensitivity calls require approval. Investigation citations are checked against recorded event IDs; invalid citations become `[unverified]`.

Run locally on macOS

1. Install and start Docker Desktop.
2. If you do not already have a `.env`, copy `.env.example` to `.env`. Keep secrets in `.env`; never commit it. Set a long random `JWT_SECRET`. Add `GROQ_API_KEY` or `GEMINI_API_KEY` only if you want cloud investigation, or use Ollama.
3. Start services: `docker compose up --build -d`.
4. Run `./demo/all_tests.sh`. Phase 2 checks ALLOW, DENY, and approval. Phase 3 trains a demo model, runs benign and attack traffic, and prints evaluation metrics. Phase 4 runs only when Ollama or a Groq/Gemini key is available.
5. Start the dashboard: `docker compose --profile ui up --build -d`. Open `http://localhost:3000`. API docs: `http://localhost:8080/swagger-ui.html`.
6. Stop with `docker compose --profile ui down`. This keeps database volume data. Avoid `down -v` unless you intend to delete the local database.

For Ollama, install it from its official distribution, start `ollama serve`, and download `llama3.1:8b` and `nomic-embed-text`. Compose connects the backend to the Mac host with `host.docker.internal`. For a free-tier cloud provider, put `GROQ_API_KEY` or `GEMINI_API_KEY` in `.env` and recreate the backend.

Settings to know

Setting	Purpose
`JWT_SECRET`	Signs dashboard login tokens. Use a long random value.
`GROQ_API_KEY` / `GEMINI_API_KEY`	Optional free-tier investigation provider credentials. Keep private.
`LLM_PROVIDER`	`auto` prefers a configured Groq/Gemini key, then local Ollama.
`NEXT_PUBLIC_API_URL`	Frontend API base URL, including `/api/v1`; compiled during the frontend build.
`DATABASE_URL`	Hosted Spring JDBC URL. Local Compose points Spring at its `db` container.
`MCP_SERVER_TOKEN`	Shared backend-to-tool-service secret; use the same random value for both hosted services.
`CORS_ALLOWED_ORIGINS`	Comma-separated web origins allowed to call the API; set the Vercel site URL when hosted.

Local Compose initializes tables from SQL files mounted in `docker-compose.override.yml`. The override also stubs Supabase-only `auth.jwt()` for local Postgres. For Supabase, apply `supabase/migrations/001_core_schema.sql` and `002_rls_policies.sql` in order. Do not duplicate the override SQL mounts in the base Compose file.

Publish to GitHub

This folder may not have Git history yet. From the project root:

```
git init -b main
git add .
git status --short
```

Confirm `.env` is ignored and `.env.example` contains placeholders only. Then:

```
git commit -m "Prepare AgentGuard platform"
git remote add origin https://github.com/YOUR-USERNAME/agentguard.git
git push -u origin main
```

Create an empty GitHub repository first. If Git needs your author identity, set `git config --global user.name` and `git config --global user.email`. If a secret was committed previously, rotate it and remove it from Git history before making the repository public.

Deploy the dashboard to Vercel

1. Import the GitHub repository in Vercel.
2. Set **Root Directory** to `frontend`.
3. Add `NEXT_PUBLIC_API_URL` as the public backend URL plus `/api/v1`, such as `https://YOUR-BACKEND.onrender.com/api/v1`.
4. Deploy and test login and every screen. `NEXT_PUBLIC_*` values are public browser configuration and get embedded during build, so redeploy after changing one.
5. Set the backend's `CORS_ALLOWED_ORIGINS` to the exact Vercel site origin (no path); comma-separated values are supported for preview and production domains.

Host the backend services

Vercel does not run this Docker Compose stack. The included `render.yaml` is a starting point for Spring, ML, and MCP services. Create a Render Blueprint from GitHub, inspect the created service names and URLs, then set the backend's `DATABASE_URL`, `DB_USER`, `DB_PASSWORD`, `JWT_SECRET`, `ML_SERVICE_URL`, and `MCP_SERVER_URL`. Add Groq/Gemini credentials only when needed. The current blueprint contains fixed service hostnames; verify or update them after Render creates the services. Free services may sleep and have resource limits.

For Supabase Postgres, copy a connection string from the Supabase dashboard's **Connect** dialog. Pick a connection mode supported by the host, replace its password placeholder, URL-encode reserved password characters, and use `jdbc:postgresql://` for Spring. Set `DB_USER` and `DB_PASSWORD` to match. Apply both migrations and verify pgvector is available. Never put database passwords or a service-role key in Vercel browser variables.

Hosted plans and free quotas change; check current terms before relying on them. A live public deployment, production credentials, domain, and demo recording are not included in this local verification.

Interview explanation

30-second version: "I built AgentGuard to govern AI agents that call tools. Every call goes through a Spring Boot policy proxy, is recorded, and is scored by a Python Isolation Forest service. Suspicious behavior opens a case. An LLM can summarize retrieved evidence, but code validates citations and a human makes decisions. I built the dashboard and local Docker demo using Java, Python, FastAPI, PostgreSQL/pgvector, and Next.js."

Walkthrough: show the architecture; register an agent; allow a web search; deny a disallowed tool and show its policy ID; require approval for a sensitive action; run the Phase 3 attack and show the case and F1; explain how invalid event citations are marked `[unverified]`.

Likely questions:

- **Why put policy before tools?** It creates one supported checkpoint for agent tool access.
- **What does Isolation Forest do?** It flags feature combinations that sit apart from learned normal behavior. The demo uses synthetic training data, so perfect demo metrics are not a production guarantee.
- **Why use an LLM?** It drafts readable context; deterministic code checks citations and a person decides what happens.
- **What if ML is unavailable?** High-sensitivity calls fail closed and require approval.
- **What would you improve?** Automated API/UI tests, safer demo credentials, a production CORS allowlist, generated hosted service URLs, persistent models, and evaluation on representative traffic with false-positive reporting.